```
\vspace*{\baselineskip}
\vspace*{\baselineskip}
\noindent\includegraphics[width=\textwidth]
{\detokenize{/workspaces/BriteTest/docs/md/../branding/Contributor_Guide.png}}

\vspace{\baselineskip}
\vspace{\baselineskip}
\noindent Document Version: 1.0\\
\noindent Runner Version: 1.0.0\\
\noindent Test Version: 1.0.0\\
```

This document defines the contribution process, coding standards, documentation rules, and versioning guidelines for contributors, reviewers, and approvers.

```
\vfill
```



```
\clearpage
\ltsetrunninghead{Preface}
\global\ltaftertoctrue
```

Preface

This document is for contributors, reviewers, and approvers who need guidance on enhancing and maintaining BriteTest.

For a list of other documents and the repository layout, see the Documentation Guide.

For a glossary of terms, see the Glossary Reference.

Document Version History

Document	Runner	Test	Date	Comment	Author/Editor
1.0	1.0.0	1.0.0	2026-06-11	Initial version.	Paul Sinclair

- The **Document** column records the document's version with the format `M.u` (Major, update).
- The **Runner** column records the Runner API version current when this document was released and is defined by its `RA_VERSION` macro.
- The **Test** column records the Test API version current when this document was released and

is defined by its `TA_VERSION` macro.
- Both the Runner API and Test API use the version format `"M.m.p"` (Major, minor, patch).
- `M` is the same for the Document, Runner, and Test versions.

The document's update version tracks released updates to this document and does not correspond to a minor or patch version. `u` increments when document changes are released without a change to `M`, and it resets to `0` when `M` is incremented.

```
\clearpage
\ltsetrunninghead{Table of Contents}
\global\ltaftertoctrue
```

Table of Contents


```
\clearpage
\pagenumbering{arabic}
\setcounter{page}{1}
\phantomsection
\label{introduction}
\ltsetrunninghead{1. Introduction}
\global\ltaftertoctrue
```

1. Introduction

This Contributor Guide defines the expectations and rules for contributing to BriteTest. It covers versioning, testing, documentation, code style, pull request, and release requirements.

The config/contibutors.md file defines the contributors and which contributors are also reviewers or approvers. Contributors should read this document before submitting changes, reviewing, or approving to ensure consistency across the code and documentation.

```
\clearpage
\phantomsection
\label{versioning-guidelines}
\ltsetrunninghead{2. Versioning Guidelines}
\global\ltaftertoctrue
```

2. Versioning Guidelines

Versioning is used for:

- API .h and .c files: `M.m.p` (major, minor, patch).
- Documentation .md files: `M.u` (major, update).

When to increment for a release:

- Update:
  - Document updated.
- Patch:
  - Bug fixes or implementation improvements.
- Minor:
  - Minor additions to the APIs (e.g., new function or macro).
  - Minor additions to the Runner Framework.
  - No breaking changes (syntax or behavior) to the APIs.
  - No breaking changes to the Runner Framework.
  - Minor refactorying of implementation.
- Major
  - Major additions to the APIs.
  - Major additions to the Runner Framework.
  - Significant refactoring of the implementation.
  - The following require incrementing the Major version but should be avoided by having opt-in or other mechanism to maintain compatibility:
    - Breaking changes (syntax or behavior) to the APIs.
    - Breaking changes to the Runner Framework.
  - If incremented, it must be updated for all files (.h, .c, and .md) to the same major value, even if the file was not otherwise modified or updated with update, minor, and patch reset to 0.

When updating the version for the Runner API:

- Update `RA_VERSION` in `runnerapi.h`.
- Update `RA_VERSION_C` in `runnerapi.c`.

When updating the version for the Test API:

- Update `TA_VERSION` in `testapi.h`.
- Update `TA_VERSION_C` in `testapi.c`.

When updating the version for a document:

- Add a new entry to the top of the Document Version History table.
- Increment `u` if the major version is not incremented.
- Reset `u` to `0` when the major version is incremented.

API compatibility guidelines:

- Do not change API signatures.
- Do not change API behavior.
- Do not change exit and return code meanings.
- Deprecate before removing.
- Provide migration guidance for major changes including how to opt-in for new features and behavior.

Significant justification is required when these guidelines cannot be followed.

```
\clearpage
\phantomsection
\label{branding}
\ltsetrunninghead{3. Branding}
\global\ltaftertoctrue
```

3. Branding

This chapter discusses choices for the brand description and name plus how to change the brand name.

3.1. Alternative Brand Descriptions

BriteTest is a lightweight framework for defining, running, and reporting tests in C/C++ projects. It provides a simple core macro-based Runner, Application Programming Interface (API), a function-based Test API, fault-tolerant execution, and clear reporting. It is ideal for small to medium C projects that need reliable testing without heavy tooling and dependencies. It can be used for unit and command-line testing.

Tightened, clarified, and made more parallel to the tone you've been using for LiteTest. Below is a refined version that reads cleaner, is more precise, and positions the framework more strongly.

BriteTest is a lightweight framework for defining, running, and reporting tests in C/C++ projects. It provides a compact macro-based Runner API, a function-driven Test API, fault-tolerant execution, and clear, structured reporting. The framework is designed for small to medium C projects that need reliable automated testing without the overhead of large toolchains or external dependencies.

BriteTest supports both unit testing and command-line driven testing, making it flexible enough for embedded utilities, libraries, and standalone executables.

Option 1 — Concise & Technical

BriteTest is a lightweight C/C++ testing framework providing a macro-based runner, function-based tests, fault-tolerant execution, and clear reporting. It enables reliable unit and command-line testing for small to medium C projects without heavy tooling dependencies.

Option 2 — Clean & Professional

BriteTest is a compact C/C++ framework for defining, running, and reporting tests. It offers a simple macro runner, function-style tests, robust fault handling, and readable output, ideal for small to medium C projects that need light-weight, reliable testing.

Option 3 — More Emphasis on Purpose

BriteTest is a minimal C/C++ test framework designed for fast, reliable unit and command-line testing. Its macro-based runner, function-driven tests, and fault-tolerant execution provide strong fundamentals for small to medium C projects without heavy dependencies.

Review: BriteTest is a lightweight framework for defining, running, and reporting tests in C/C++

projects. It provides a simple core macro-based Runner Application Programming Interface (API), a function-based Test API, fault-tolerant execution, and clear reporting. It is ideal for small to medium C projects that need reliable testing without heavy tooling and dependencies. It can be used for unit and command-line testing.

Takeaway: Your description of BriteTest is solid, but it can be tightened, clarified, and made more parallel to the tone you've been using for LiteTest. Below is a refined version that reads cleaner, is more precise, and positions the framework more strongly.

Refined Review of BriteTest

BriteTest is a lightweight framework for defining, running, and reporting tests in C/C++ projects. It provides a compact macro-based Runner API, a function-driven Test API, fault-tolerant execution, and clear, structured reporting. The framework is designed for small to medium C projects that need reliable automated testing without the overhead of large toolchains or external dependencies.

BriteTest supports both unit testing and command-line driven testing, making it flexible enough for embedded utilities, libraries, and standalone executables.

Takeaway: Here are three polished, README-ready project descriptions under 350 characters, each with a slightly different tone. All stay formal and suitable for a title-page abstract.

Option 1 — Concise & Technical

BriteTest is a lightweight C/C++ testing framework providing a macro-based runner, function-based tests, fault-tolerant execution, and clear reporting. It enables reliable unit and command-line testing without external tooling or heavy dependencies.

Option 2 — Clean & Professional

BriteTest is a compact C/C++ framework for defining, running, and reporting tests. It offers a simple macro runner, function-style tests, robust fault handling, and readable output, ideal for small to medium projects needing dependable testing with minimal overhead.

Option 3 — More Emphasis on Purpose

BriteTest is a minimal C/C++ test framework designed for fast, reliable unit and command-line testing. Its macro-based runner, function-driven tests, and fault-tolerant execution provide clear reporting without the weight of large toolchains or dependencies.

Takeaway: Here are three polished, README-ready project descriptions under 350 characters, each with a slightly different tone. All stay formal and suitable for a title-page abstract.

Option 1 — Concise & Technical

BriteTest is a lightweight C/C++ testing framework providing a macro-based runner, function-based tests, fault-tolerant execution, and clear reporting. It enables reliable unit and command-line testing without external tooling or heavy dependencies.

Option 2 — Clean & Professional

BriteTest is a compact C/C++ framework for defining, running, and reporting tests. It offers a simple macro runner, function-style tests, robust fault handling, and readable output, ideal for small to medium projects needing dependable testing with minimal overhead.

Option 3 — More Emphasis on Purpose

BriteTest is a minimal C/C++ test framework designed for fast, reliable unit and command-line testing. Its macro-based runner, function-driven tests, and fault-tolerant execution provide clear reporting without the weight of large toolchains or dependencies.

Takeaway: Here are three refined, README-ready descriptions (≤350 characters) that subtly echo bright, clarity, lightness, and precision—without being cheesy or literal. Each keeps a formal tone suitable for a title-page abstract.

Option 1 — “Bright + Clear” (clean, crisp)

BriteTest is a lightweight C/C++ testing framework focused on clarity and reliability. It provides a macro-based runner, function-style tests, fault-tolerant execution, and clear reporting—ideal for projects that need dependable testing with minimal overhead.

Option 2 — “Bright + Minimal” (emphasizes lightness)

BriteTest is a minimal, fast C/C++ test framework offering a simple macro runner, function-based tests, resilient execution, and clear, concise reports. It brings bright, low-overhead testing to small and medium projects without external tooling.

Option 3 — “Bright + Purposeful” (subtle thematic resonance)

BriteTest is a lightweight C/C++ framework designed for clear, dependable testing. Its macro runner, function-driven tests, and fault-tolerant execution deliver bright, readable reports and reliable automation without heavy dependencies.

If you want, I can tune one further to feel more energetic, more formal, or more brand-forward, such as emphasizing the “brite/bright” theme more strongly through tone or rhythm. You can choose one to refine: Option 1, Option 2, or Option_3.

Takeaway: Here are three polished, README-ready project descriptions under 350 characters, each with a slightly different tone. All stay formal and suitable for a title-page abstract.

Option 1 — Concise & Technical

BriteTest is a lightweight C/C++ testing framework providing a macro-based runner, function-based tests, fault-tolerant execution, and clear reporting. It enables reliable unit and command-line testing without external tooling or heavy dependencies.

Option 2 — Clean & Professional

BriteTest is a compact C/C++ framework for defining, running, and reporting tests. It offers a simple macro runner, function-style tests, robust fault handling, and readable output, ideal for small to medium projects needing dependable testing with minimal overhead.

Option 3 — More Emphasis on Purpose

BriteTest is a minimal C/C++ test framework designed for fast, reliable unit and command-line testing. Its macro-based runner, function-driven tests, and fault-tolerant execution provide clear reporting without the weight of large toolchains or dependencies.

If you want, I can refine one further or tune it for marketing tone, academic tone, or ultra-minimal length.

Takeaway: Here are three refined, README-ready descriptions (≤350 characters) that subtly echo bright, clarity, lightness, and precision—without being cheesy or literal. Each keeps a formal tone suitable for a title-page abstract.

Option 1 — “Bright + Clear” (clean, crisp)

BriteTest is a lightweight C/C++ testing framework focused on clarity and reliability. It provides a macro-based runner, function-style tests, fault-tolerant execution, and clear reporting—ideal for projects that need dependable testing with minimal overhead.

Option 2 — “Bright + Minimal” (emphasizes lightness)

BriteTest is a minimal, fast C/C++ test framework offering a simple macro runner, function-based tests, resilient execution, and clear, concise reports. It brings bright, low-overhead testing to small and medium projects without external tooling.

Option 3 — “Bright + Purposeful” (subtle thematic resonance)

BriteTest is a lightweight C/C++ framework designed for clear, dependable testing. Its macro

runner, function-driven tests, and fault-tolerant execution deliver bright, readable reports and reliable automation without heavy dependencies.

If you want, I can tune one further to feel more energetic, more formal, or more brand-forward, such as emphasizing the “brite/bright” theme more strongly through tone or rhythm. You can choose one to refine: Option 1, Option 2, or Option_3.

Takeaway: Here are three more energetic, still-formal, ≤350-character README abstracts that weave in the brite/bright theme with momentum and confidence.

Option 1 — Energetic + Crisp

BriteTest is a fast, lightweight C/C++ testing framework built for clarity and momentum. Its macro runner, function-based tests, and fault-tolerant execution deliver bright, reliable reports with minimal overhead—ideal for projects that want testing that stays out of the way.

Option 2 — Energetic + Punchy

BriteTest brings bright, low-overhead testing to C/C++ projects. With a simple macro runner, clean function-style tests, resilient execution, and sharp reporting, it keeps test cycles quick and clear for teams that want speed without sacrificing reliability.

Option 3 — Energetic + Brand-Forward

BriteTest is a lightweight C/C++ test framework engineered for bright, fast, dependable feedback. Its macro runner, function-driven tests, and robust fault handling keep results clear and cycles tight—perfect for projects that value clarity and pace.

If you want, I can tune one to be more technical, more brand-loud, or even tighter: Refine Option 1, Refine Option 2, or Refine Option 3.

I like this one since it invokes bright without being obvious by explicitly saying bright which is already in the brand name.

Got it — you’re gravitating toward the version that feels bright without literally saying it. That’s exactly the right instinct for a brand like BriteTest: let the tone carry the theme, not the wording.

Here’s a tightened, energetic version in that same style, still under 350 characters and keeping the “bright without saying bright” vibe:

Polished Energetic Abstract

BriteTest is a lightweight C/C++ testing framework built for clarity and speed. Its macro runner, function-driven tests, and fault-tolerant execution keep feedback sharp and cycles fast—ideal for projects that want dependable testing with minimal overhead.

Got it — you’re gravitating toward the version that feels bright without literally saying it. That’s exactly the right instinct for a brand like BriteTest: let the tone carry the theme, not the wording.

Here’s a tightened, energetic version in that same style, still under 350 characters and keeping the “bright without saying bright” vibe:

Polished Energetic Abstract

BriteTest is a lightweight C/C++ testing framework built for clarity and speed. Its macro runner, function-driven tests, and fault-tolerant execution keep feedback sharp and cycles fast—ideal for projects that want dependable testing with minimal overhead.

If you want, I can tune it to lean slightly more toward technical precision, brand voice, or even more energy Takeaway: Here is a technically precise, energetic refinement of Option 2. It keeps the “brite without saying bright” tone, removes soft language, and sharpens the engineering vocabulary—still under 350 characters. Technically Precise Version

BriteTest is a compact C/C++ framework for defining, running, and reporting tests. It provides a macro-driven runner, clean function-style test definitions, resilient fault isolation, and clear output —well-suited for small to medium projects requiring fast, dependable, low-overhead testing.

Here are my top three versions based on clarity, technical precision, energy, and how well they fit a README title-page abstract. All three stay under 350 characters and keep the “brite without saying bright” tone.

Top Choice 1 — Technically Precise + Energetic

BriteTest is a compact C/C++ framework for defining, running, and reporting tests. It provides a macro-driven runner, clean function-style test definitions, resilient fault isolation, and clear output —well-suited for small to medium projects requiring fast, dependable, low-overhead testing.

Top Choice 2 — Clean, Energetic, Balanced

BriteTest is a compact C/C++ framework for defining, running, and reporting tests. It offers a simple macro runner, clean function-style tests, robust fault handling, and clear output—ideal for small to medium projects that need fast, dependable testing with minimal overhead.

Top Choice 3 — Slightly More Dynamic

BriteTest is a lightweight C/C++ framework for defining, running, and reporting tests. Its macro runner, function-driven tests, and fault-tolerant execution deliver clear, reliable results—suited for small to medium projects that want fast, dependable testing without heavy tooling.

Here are three tightened, technically precise, energetic versions of your abstract that now explicitly convey ease of use without adding fluff or breaking the “brite-without-saying-bright” tone. All stay under 350 characters.

Option A — Technical + Ease of Use (balanced)

BriteTest is a compact C/C++ framework for defining, running, and reporting tests. It provides an easy-to-use macro runner, clean function-style test definitions, resilient fault isolation, and clear output—ideal for small to medium projects needing fast, dependable, low-overhead testing. Option B — More Energetic + Ease of Use

BriteTest is a compact C/C++ framework for defining, running, and reporting tests. Its easy-to-use macro runner, clean function-style tests, robust fault isolation, and clear output make it well-suited for small to medium projects that need fast, dependable testing with minimal overhead.

Option C — Most Technical + Ease of Use

BriteTest is a compact C/C++ framework for defining, running, and reporting tests. It offers an easy-to-use macro-driven runner, concise function-style test definitions, strong fault isolation, and clear reporting—ideal for small to medium projects requiring fast, reliable, low-overhead testing.

3.2. Alternative Brand Names

TODO - add these to table:

- briteTest (bT)
- litetest (lT)
- openTest (oT)
- canaryRunner (cR)
- CanaryTestRunner (CTR)
- canaryTestRunner (cTR)
- canaryTestrunner (cTr)

These names are fallback options if a naming conflict requires a brand/project rename.

These notes are an informal naming screen only. They are based on how generic, descriptive, or commonly used the terms appear in software and testing. They are not a trademark search or legal clearance.

Name	Abbrev.	Informal conflict note	Likelihood
LiteTest	LT	Clear and close to the project's lightweight positioning, but both `lite` and `test` are common software terms, so overlap with	Medium

		existing package or tool names is plausible.	
briteTest	bT	Strong fit for the project brand with canary monogram, distinctive camelCase format, and clear semantic meaning. Low conflict likelihood.	Low
canaryTest	cT	Strong fit for the canary theme, but `canary` and `canary testing` are already common software terms, which makes the name less distinctive.	Higher
clearTest	cT	Readable and descriptive, but the name is broad and likely to overlap with existing testing or QA branding.	Medium
tinyTest	tT	Good match for a lightweight framework, but it is fairly descriptive and similar in shape to other small-test framework names.	Medium
swiftTest	sT	Memorable, but `Swift` has strong existing association with Apple's Swift ecosystem, which could create confusion.	Higher
quickTest	qT	Familiar and easy to say, but `QuickTest` has long-standing use in software testing and QA tooling.	Higher
compactTest	cT	Fits the lightweight positioning and is somewhat more distinctive than `CoreTest` or `ClearTest`, though still descriptive.	Medium-Low
fastTest	fT	Strong performance-oriented signal, but `fast` and `test` are both generic terms and likely to overlap with existing tooling names.	Medium-High

3.3. Brand Name and Tagline Replacement

This workflow describes how to replace the brand name, tagline, and other phrases throughout the repository (documentation, monograms, and logos) using automated scripts.

Prerequisites

The replacement workflow depends on a configuration file and two scripts:

- **Configuration**: `docs/branding/brand.md`:
  - Defines old-to-new phrase replacements.
  - For a brand name change, its brand abbreviation is auto-generated from the first letter of each word concatenated into the brand name.
  - Example: `- **Replace**: LiteTest = briteTest` generates replacements for both "LiteTest" to "briteTest" (brand name) and, for monograms and logos, "LT" to "bT" (brand abbreviation)
- **Scripts**:
  - `scripts/bin/updatelogos`: Updates SVG files in `docs/branding/` with phrase and abbreviation replacements, then regenerates PNG files
  - `scripts/bin/replacephrases`: Updates markdown files (`*.md`) in `docs/` with phrase replacements only

Workflow Steps

1. Choose a replacement brand name and/or tagline plus any other phrases that need changing.
2. Update the brand configuration file `docs/branding/brand.md` to define the replacements (see this file for examples of previous changes):

```
### <change title>
\<Reason for the change.\>

 - \*\*Replace\*\*: old_phrase = new_phrase
...

3, The file supports multiple replacements (one per line):

   - Each replacement is validated to prevent circular references (an old phrase
```

```
    cannot match any new phrase).

4. Update SVG files and regenerate PNG files:

   - Run the `updatelogos` script:

```bash
scripts/bin/updatelogos
```

5, The script performs the following:

- Parses `docs/branding/brand.md` for new phrase replacements (occurring before the first "**Completed**: <datetime>." line in the file.
- Validates the configuration to prevent infinite loops (circular references).
- Applies both brand namd and abbreviation replacements to all SVG files in `docs/branding/`.
- Regenerates all PNG files from the updated SVGs using `scripts/bin/genpng`.
- Generates a detailed report of replacements and file modifications.
- Review the output and documentation for accuracy, then commit the changes.

6. Update project identifier and repository name (optional:
    - Update references in CI/CD workflows, and external links.
7. Validate and test

- Run `make run` to ensure tests pass.
- Run `make pdf` or the PDF generation script to verify branding updates in generated documents.
- Perform a repository-wide text search for stale old-brand references.
- Verify that all SVG taglines have been updated correctly.

8. Commit and ship
    - Commit SVG/PNG changes with a clear commit message.
    - Commit markdown changes with a clear commit message.
    - Optionally, add a note to `README.md` in root directory documenting the brand transition.
    - Consider a separate release or major version bump if the replace is significant.

```
\clearpage
\phantomsection
\label{documentation-guidelines}
\ltsetrunninghead{4. Documentation Guidelines}
\global\ltaftertoctrue
```

4. Documentation Guidelines

This section exists here to ensure contributors do not overlook the requirement for parallel structure and consistent writing patterns across all documentation in all types of files (e,g., `.md`, `.h`, `.c`, `.yml`, `.dh`, `Makefile`, etc.).

1. General
    - Root `README.md` must remain short and onboarding-focused.
    - A User Guide contains conceptual explanations and examples.
    - A API Reference contains public API definitions only.
    - An internal guide or reference must not leak into public docs.
    - Update documentation when enhancing macros, behavior, or report format.
    - For branding assets in `docs/branding/`: `*.svg` files are the source of truth and `*.png` files are generated from SVG using `scripts/bin/genpng`.
2. Tone
    - Technical, precise, and neutral.
    - Minimal marketing language.
    - Prefer clarity over cleverness.
```

3. Formatting
    - Use backticks for code identifiers.
    - Use fenced code blocks for file trees, examples, and commands.
    - Keep line lengths reasonable for GitHub rendering.
    - Use boldface for a term or phrase when defining it
    - Ccnform to formatting styles existing in the documentation.
4. Writing Guidelines
    - Define terms once, and then use them consistently.
    - Avoid synonyms for technical concepts (e.g., always "update version," never "revision").
    - Keep paragraphs short.
    - Use lists for enumerations.
5. Collapsible Sections in `.md` Files
    - Use collapsible chapters, sections and subsections to keep the document readable while still accommodating large amounts of technical detail.
    - Collapsing sections allows readers to scan the structure and expand only what they need.
    - This keeps the document manageable, avoids overwhelming readers with unrelated detail, and makes the document easier to navigate.
6. Style Consistency

To keep documentation clear and easy to read, maintain **parallel structure** within lists and related sentences:

- Start list items with the same part of speech (typically a verb).
- Keep grammatical patterns consistent across bullets.
- Avoid mixing styles such as "Keep paragraphs short" with "Using lists for enumerations.".
- Rewrite items as needed so the list reads smoothly and uniformly.

This guideline applies to all documentation (`.md` files).

```
\clearpage
\phantomsection
\label{code-guidelines}
\ltsetrunninghead{5. Code Guidelines}
\global\ltaftertoctrue
```

5. Code Guidelines

- C99.
- POSIX.1-2001 APIs only.
- Keep `runnerapi.h` and `testapi.h` each self-contained.
- Keep `runnerapi.c` and `testapi.c` implementation-only.

```
\clearpage
\phantomsection
\label{testing-requirements}
\ltsetrunninghead{6. Testing Requirements}
\global\ltaftertoctrue
```

6. Testing Requirements

- Build and run tests from the repository root:

```
make run
```

- Ensure report formatting changes are reflected in documentation examples.
- Keep test code aligned with the current version of `runnerapi.h` and `testapi.h`.

```
\clearpage
```

```
\phantomsection
\label{M#7-making-modifications-in-a-branch}
\ltsetrunninghead{7. Making Modifications in a Branch}
\global\ltaftertoctrue
```

7. Making Modifications in a Branch

- Define modifications that are focused (not mixing unrelated changes), logically grouped, and well-scoped.
- Define an appropriate name for your changes to use as the branch nmme.
- Obtain preliminary approval and target release for the modifications and branch name.
- Create a branch using your branch name.
- Modify the code in the branch as needed.
- Modify the documentation in the branch as needed in parallel with any code changes.
- Ensure the modifications in the branch are still focused (not mixing unrelated changes), logically grouped, and well-scoped.
- Ensure modifications follow the Documentation Guidelines and the Code Guidelines,
- Ensure version numbers are updated for modified versioned files as needed per the Versioning Guidelines.
- Include test coverage for new behavior and modifications.
- Ensure tests pass: run `make run` from the repository root.

```
\clearpage
\phantomsection
\label{pull-request-pr}
\ltsetrunninghead{8. Pull Request (PR)}
\global\ltaftertoctrue
```

8. Pull Request (PR)

A **Pull Request (PR)** is a formal proposal to merge your code changes into the `main` branch of the repository. It's the mechanism that enables code review, quality assurance, and collaborative development before changes become part of the official codebase.

Pull Request Workflow

Step 1: Prepare Your Changes

Before submitting a PR:

- Ensure changes for PR are focused (not mixing unrelated changes), logically grouped, and well-scoped.
- Update documentation as needed in parallel with code changes.
- Ensure changes follow the Documentation Guidelines and Code Guidelines,
- Update version numbers of modified versioned files as needed per the Versioning Guidelines.
- Include test coverage for new behavior and modifications.
- Ensure tests pass: run `make run` from the repository root.

Step 2: Create the Pull Request

- Push your branch to the repository
- Open a PR on GitHub with a clear title and description
- Reference any related issues or discussions
- Ensure the PR scope is minimal and logically grouped

Step 3: Code Review and Approval

- An **approver** (contributor with commit access) and, optionally, other contributors review the PR to verify:
  - Tests pass and there are no regressions,
  - Versioning rules are enforced,
  - Documentation consistency is maintained,

    - Changes align with code and contribution guidelines,
    - The Approver or a reviewing contributor may equest changes.
- The approver does one of the following:
    - Approves the PR for the next release - continue with Step 4.
    - Rejects the PR.
    - Defers the PR for a subsequent release. The contributor may resubmit the PR after the next release is committed or when advised by the approver.

Step 4: Merge into Main

- Once approved, the PR is merged into the `main` branch.
- Changes are now part of the official codebase but not yet released.
- The branch can be deleted after merging.

```
\clearpage
\ltsetrunninghead{9. Release}
\global\ltaftertoctrue
```

9. Release

A **Release** is the act of publishing a specific version for public use. It packages approved and merged changes into a versioned, immutable snapshot that users can download and depend on.

Release Workflow

Step 1: Prepare the Release

Before releasing:

- Prepare a preliminary release note for user that summarizes the changes.
- Verify if release qualifies as a major, minor, patch, or documentation-only,
- Verify modified versioned files have the correct versions consistently across all files per the Version Guidelines.
- Ensure API compatibility guidelines are followed.
- Document all breaking changes with migration guidance.
- Test thoroughly.
- Finalize a clear and concise release note.

Step 2: Validate Changes

- Run `make run` to ensure all tests pass
- Run `make pdf` to verify branding updates in generated documents
- Perform repository-wide text search for stale old-version references
- Ensure all changes align with API compatibility guidelines:
    - Do not change API signatures.
    - Do not change API behavior.
    - Do not change exit and return code meanings.
    - Deprecate before removing.
    - Provide migration guidance for major changes.

Step 3: Create Release Commit

- Create a commit with clear message documenting the release.
- Include version updates for all files.
- Include updated documentation and version history tables.
- Optionally, update `README.md` with release notes.

Step 4: Tag and Publish

- Tag the commit with the new version (e.g., `v1.2.3`).
- Push the tag to the repository.

- Create a GitHub release with the release note.
- Make the release publicly available for download.

```
\clearpage
\phantomsection
\label{glossary}
\ltsetrunninghead{10. Glossary}
\global\ltaftertoctrue
```

10. Glossary

For a glossary of terms generally used in the documentation, see the Glossary Reference.

Contributor-Specific Terms:

- **Approver**: A contributor with commit access who reviews pull requests, enforceS versioning rules, and ensures documentation consistency.
- **Breaking Change**: A change that alters public API behavior, removes or renames macros, changes return semantics, or requires a major version bump.
- **Contributor**: A person submitting code, documentation, fixes, or improvements.
- **Deprecated**: A public API element marked for removal in a future major version, requiring documentation updates and migration guidance.
- **Documentation Update**: Any modification to versioned documentation requires incrementing the update version or, if an API major version is incremented, incrementing the major version per the Versioning Guidelines.
- **Internal API Change**: A change to an API's internals that does not affect public API users but may require updates to an Internal Guide or Internal Reference.
- **Major Increment**: A structural or conceptual overhaul that causes a breaking change.
- **Public API Change**: Any modification to the Runner Framework/API or rhe Test API that requires a version bump.
- **Pull Request**:
- **Release**:
- **Test Coverage Requirement**: The expectation that all code changes include new tests (if adding behavior), updated tests (if modifying behavior), and no regressions.
- **Versioned Files**: Files that include a version and version history.